Penetration Testing Lab Report

TryHackMe: Pickle Rick

CEH Practical Module — Web Application Penetration Testing

Prepared by	Manoj
Lab / Room	TryHackMe – Pickle Rick
Category	Web Application Security (CEH – Web module)
Target IP	10.48.132.224 (later reassigned 10.49.130.59)
Objective	Gain access to the target machine and retrieve 3 hidden “ingredient” flags
Attacker Machine	Kali Linux 2026.2 (VMware)

1. Objective

This report documents the steps performed while completing the “Pickle Rick” room on TryHackMe as part of the Web Application module of the CEH course. The goal of the room is to exploit a vulnerable web application to obtain command execution on the target and retrieve three hidden “ingredient” flags, simulating a full black-box web penetration test: reconnaissance, enumeration, exploitation, and privilege escalation.

2. Scope and Tools Used

- Attacker machine: Kali Linux 2026.2 (VMware)
- Target: TryHackMe “Pickle Rick” VM
- Nmap – port and service scanning
- Firefox – web browsing, view-source, and manual testing
- Gobuster – directory/file enumeration
- Built-in web “Command Panel” – used as the command execution interface after login

3. Methodology / Steps Performed

3.1 Reconnaissance – Port Scanning

An Nmap scan was run against the target to identify open ports and running services:

```
nmap -sV -p- 10.48.132.224
```

Result:

- Port 22/tcp – open – SSH (OpenSSH 8.2p1 Ubuntu)
- Port 80/tcp – open – HTTP (Apache httpd 2.4.41 Ubuntu)

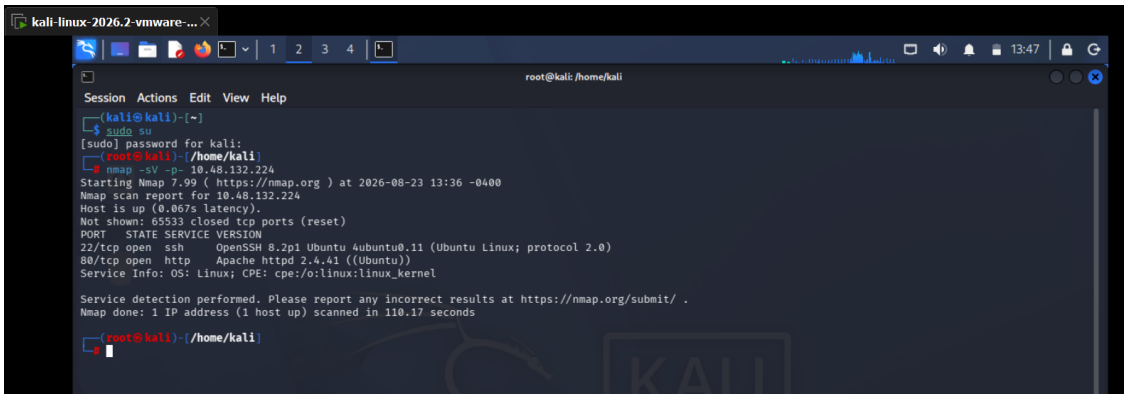


Figure 1: Nmap scan showing ports 22 (SSH) and 80 (HTTP) open.

Since port 80 was open, the IP address was pasted directly into Firefox. Only a static “Help Morty!” page was visible, with no obvious links or forms.

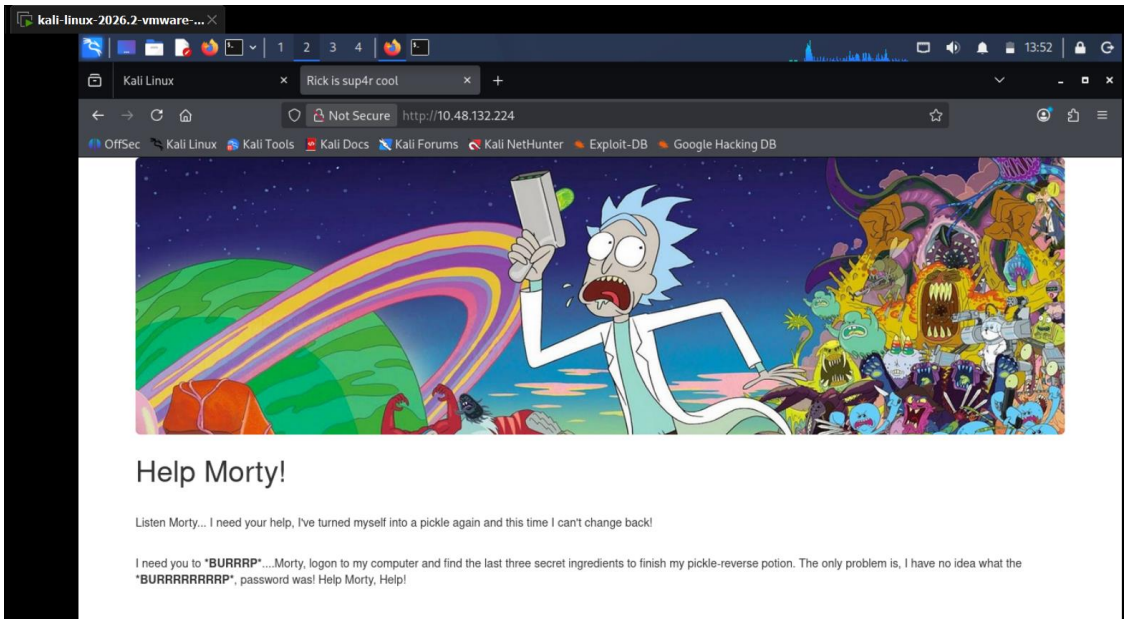


Figure 2: Initial web page at http://10.48.132.224 — "Help Morty!" page.

3.2 Enumeration – Directory Brute-Forcing

As the visible page had nothing else to interact with, Gobuster was used to brute-force hidden directories and files on the web server:

```
gobuster dir -u http://10.48.132.224 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 20 -x html,txt,php
```

This revealed several pages that were not linked from the homepage:

- index.html – 200 OK
- login.php – 200 OK
- assets/ – 301 redirect
- portal.php – 302 redirect to /login.php (protected page)
- robots.txt – 200 OK

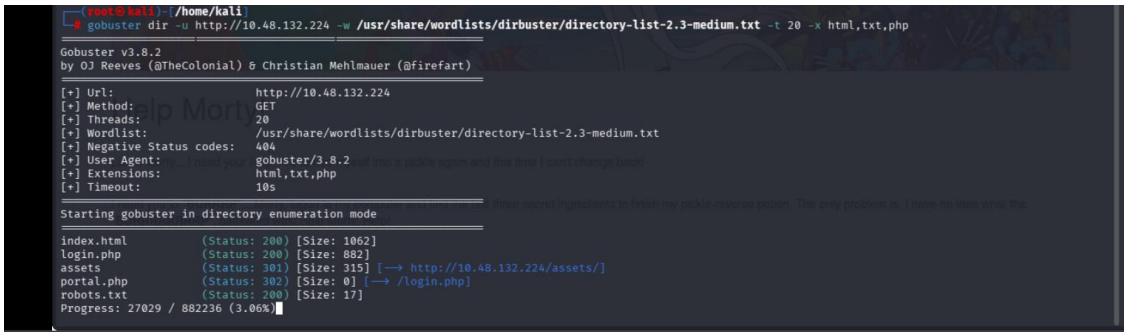


Figure 3: Gobuster output listing index.html, login.php, assets, portal.php and robots.txt.

3.3 Information Disclosure – Credential Discovery

Each discovered page was checked manually:

- Viewing the page source of the homepage (Ctrl+U / view-source:) revealed an HTML comment left by the developer containing a username.

- robots.txt returned a text string that turned out to be the matching password.

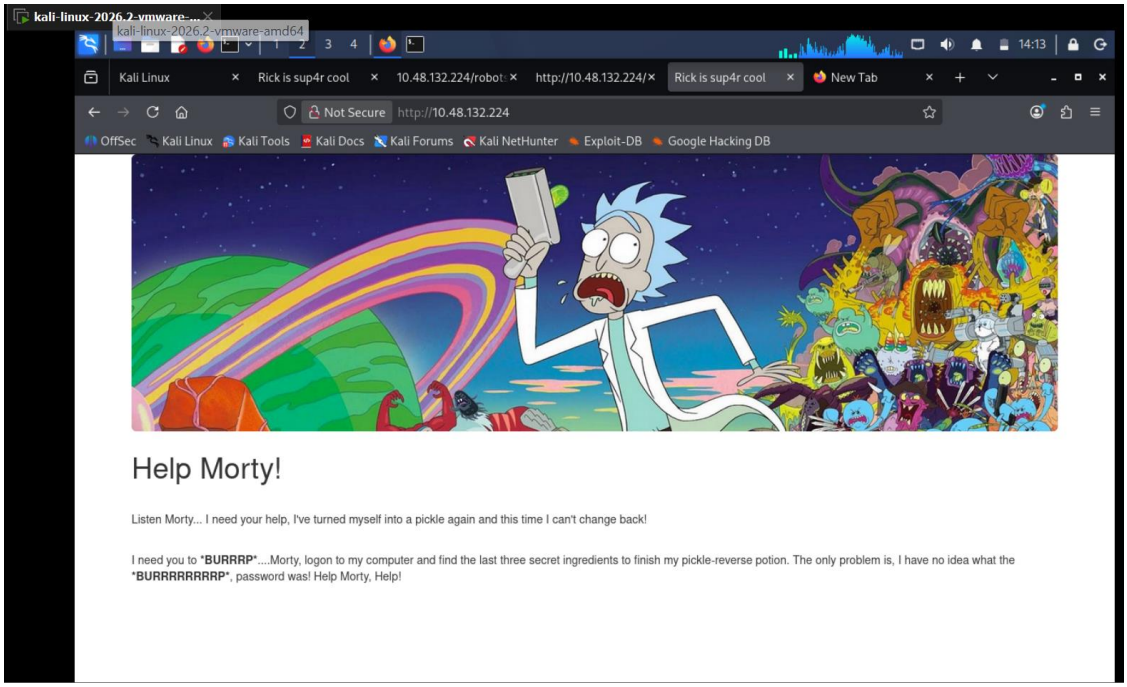


Figure 4: Homepage revisited before checking the page source.

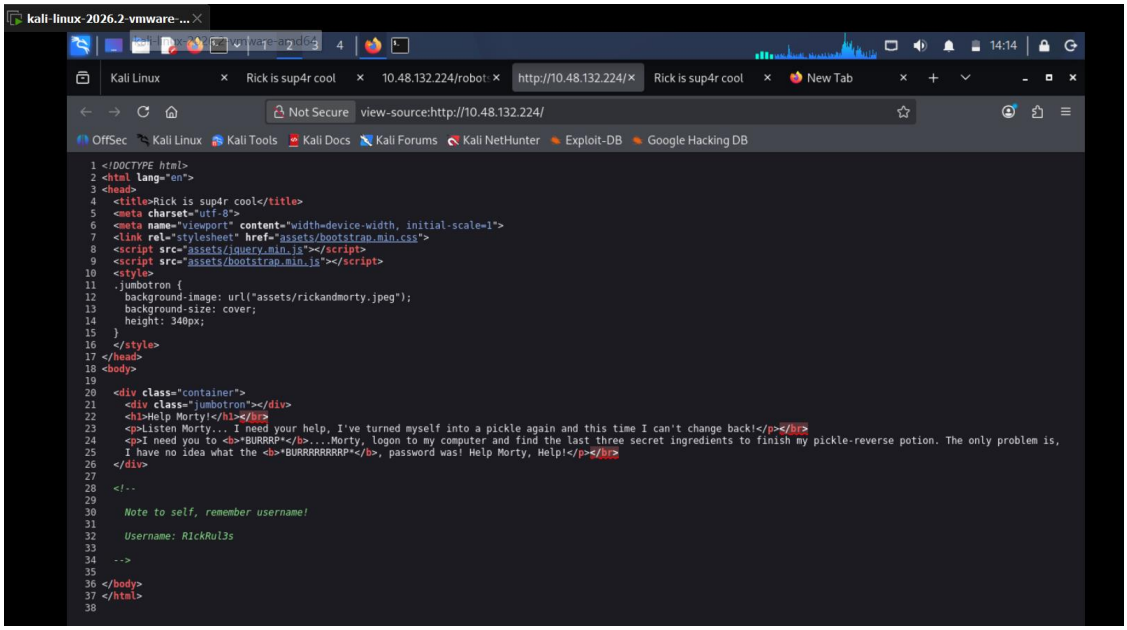


Figure 5: view-source of the homepage — HTML comment disclosing username "R1ckRul3s".

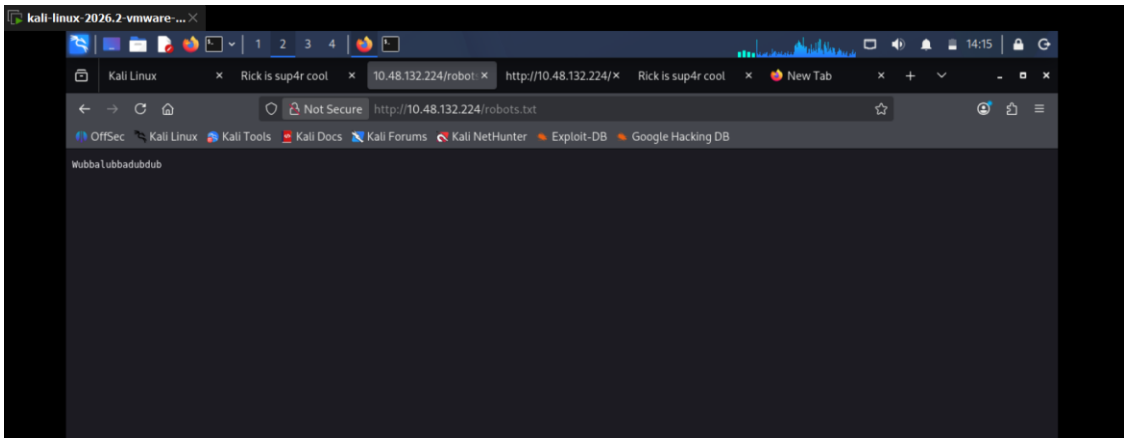


Figure 6: Contents of /robots.txt — disclosed string "Wubbalubbadubdub", used as the password.

These two values (username: R1ckRul3s, password: Wubbalubbadubdub) were disclosed to any unauthenticated visitor simply by viewing page source and a robots.txt file — this is the Information Disclosure vulnerability. Both were entered into login.php.

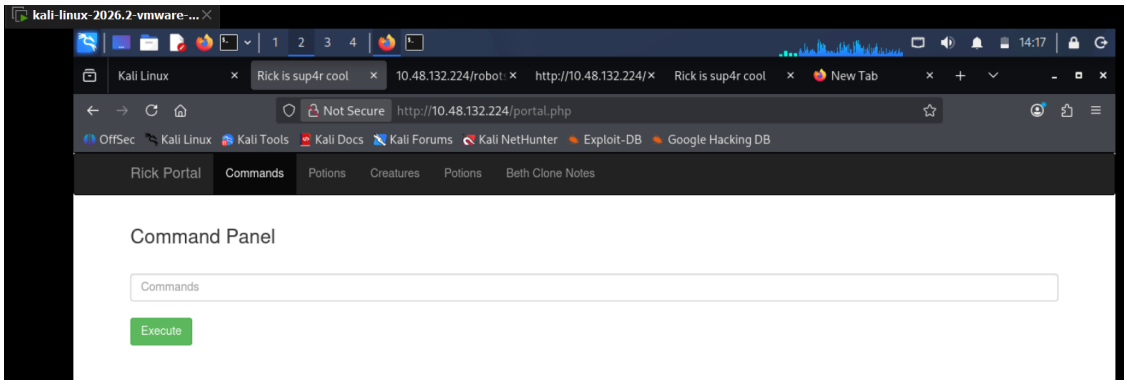


Figure 7: login.php successfully authenticated and redirected to portal.php, showing the Command Panel.

Working: Credentials R1ckRul3s / Wubbalubbadubdub authenticated successfully on login.php

3.4 Exploitation – Command Injection via Command Panel

portal.php exposes a "Command Panel" text box that executes operating system commands on the server and returns the raw output directly in the browser — i.e. an OS command injection vector with no authentication-independent input sanitisation. This was used to explore the filesystem and locate the three ingredient flags.

3.5 Flag 1 – Web / Application Directory

Running ls in the command panel from the web root listed the files present, including a file named Sup3rS3cretPickl3Ingred.txt:

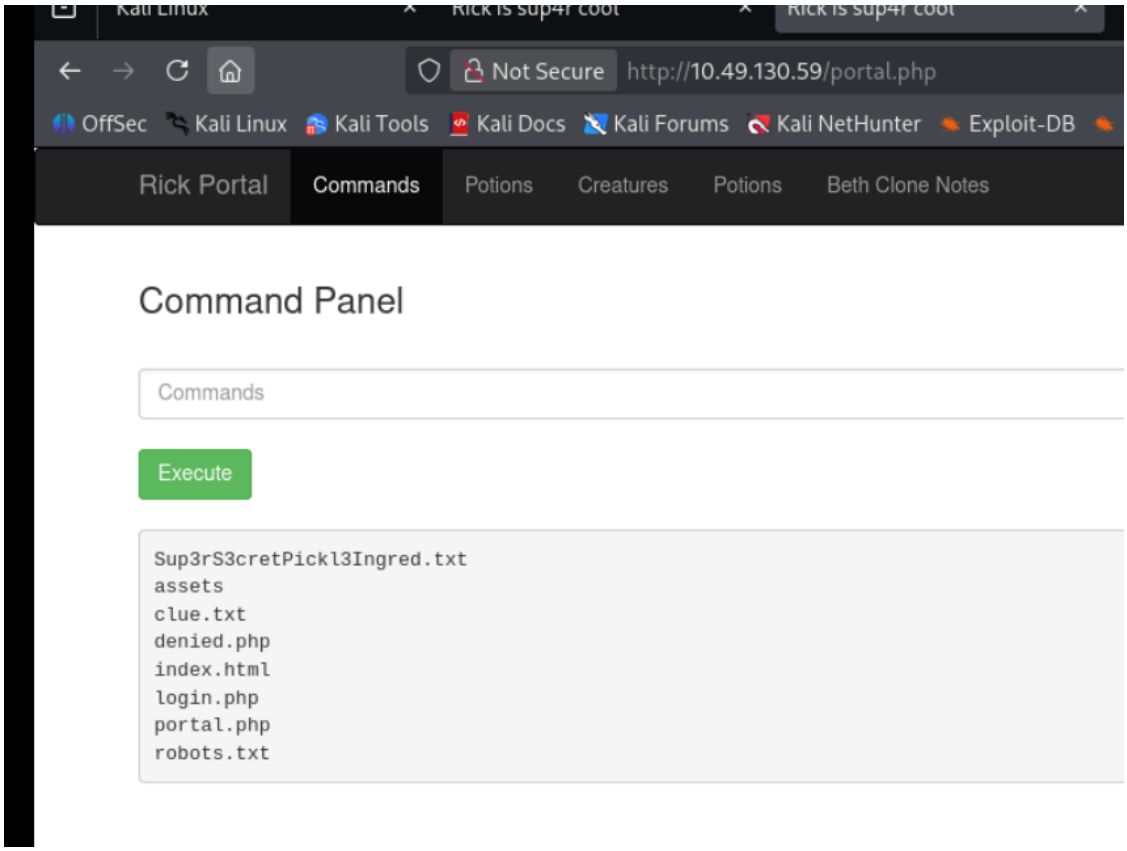


Figure 8: Command Panel output of 'ls' listing Sup3rS3cretPickl3Ingred.txt, assets, clue.txt, denied.php, index.html, login.php, portal.php, robots.txt.

Not working: cat Sup3rS3cretPickl3Ingred.txt — command was blocked/filtered by the application

Working: less Sup3rS3cretPickl3Ingred.txt — used as an alternative to the blocked 'cat' command

The file was located in /var/www/html and, once read, contained the first ingredient:

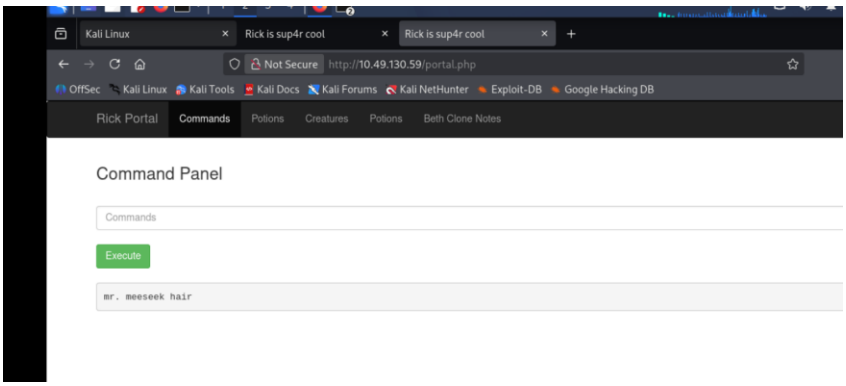


Figure 9: First ingredient retrieved — 'mr. meeseek hair'.

- **Flag 1 (Web directory): "mr. meeseek hair"**

3.6 Flag 2 – User Flag (/home/rick)

To look for local users on the box, the /home directory was listed:

```
ls -la /home
```

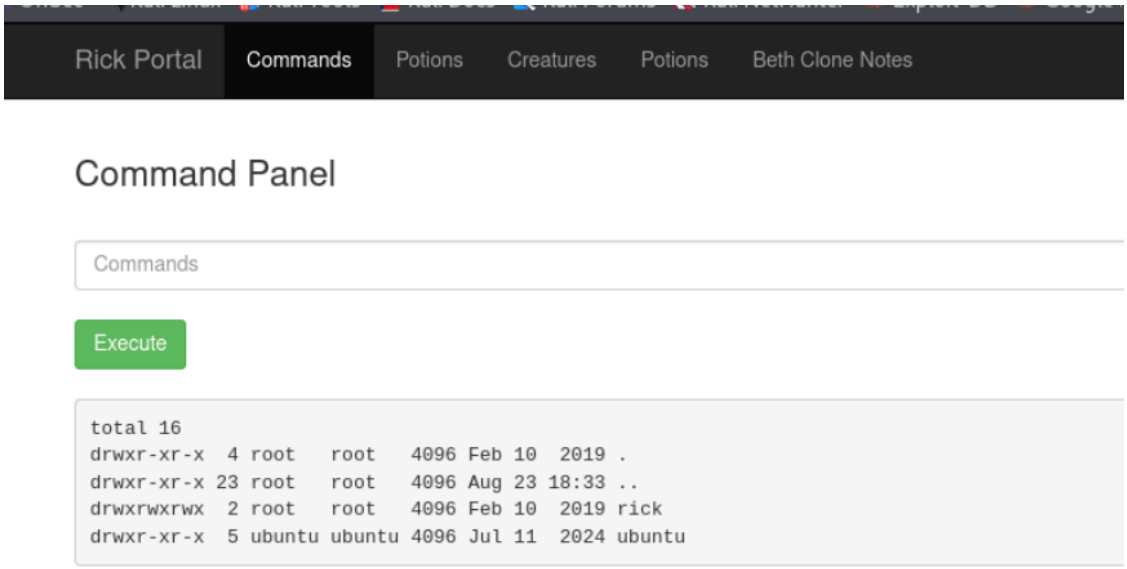


Figure 10: 'ls -la /home' output showing two users: rick and ubuntu.

Two user directories were found: rick and ubuntu. Listing rick's home directory revealed a file called "second ingredients" (note the space in the filename):

```
ls -la /home/rick
```

Not working: less /home/rick/second ingredients — failed because the filename contains a space and was not quoted

Working: less "/home/rick/second ingredients" — quoting the path handled the space correctly

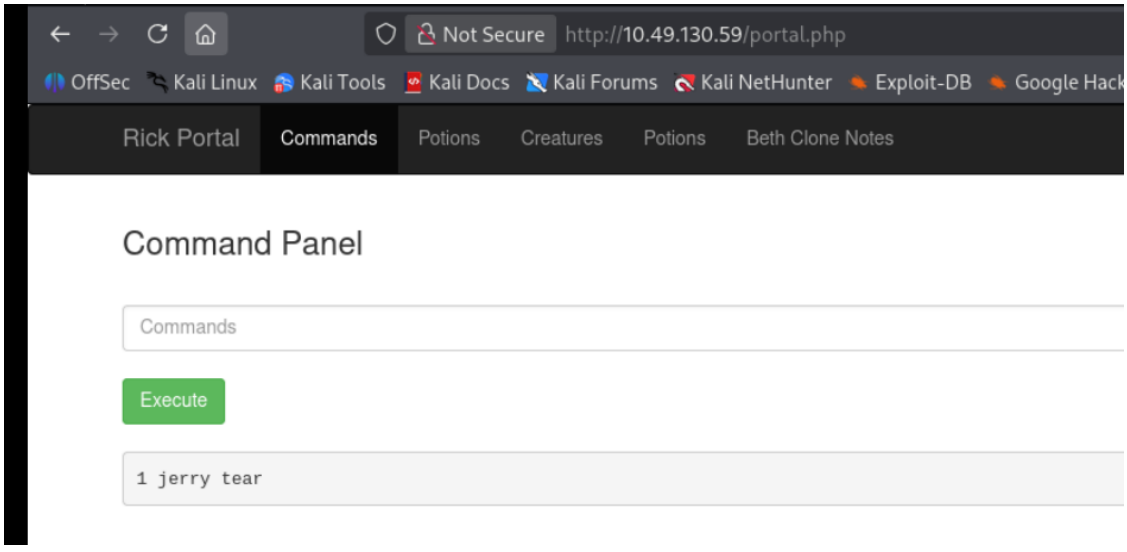


Figure 11: Second ingredient retrieved — '1 jerry tear'.

- **Flag 2 (User flag): "1 jerry tear"**

3.7 Flag 3 – Root Flag / Privilege Escalation

The final ingredient was expected to be under /root. A direct listing attempt failed due to insufficient privileges as the current user (www-data):

Not working: ls -la /root — permission denied for the www-data user

Privilege escalation was therefore attempted using sudo, and the current user was found to have unrestricted sudo rights:

Working: sudo ls -la /root — succeeded and listed root's home directory

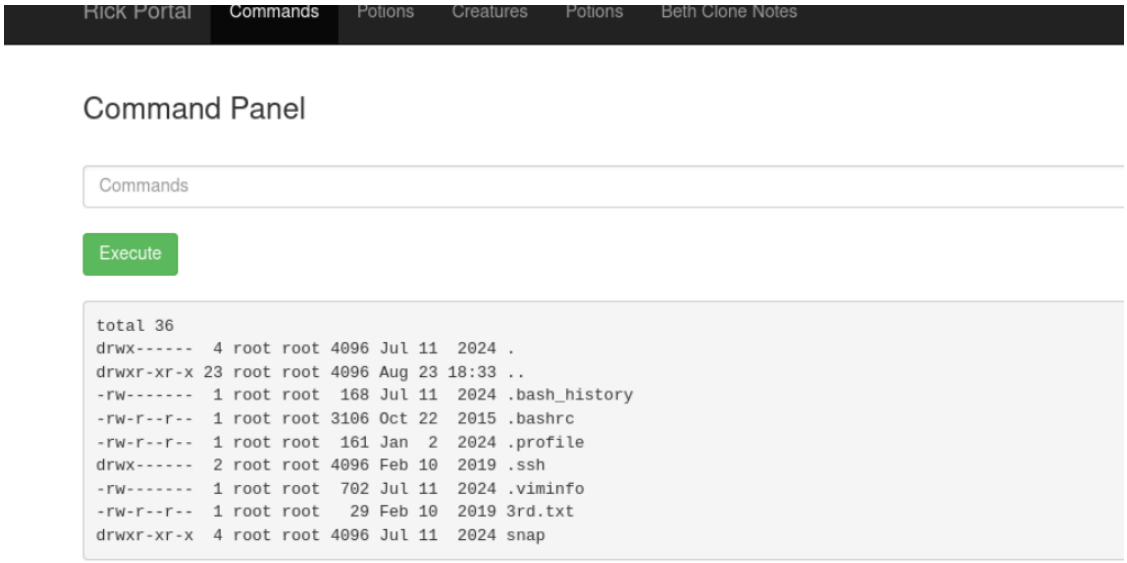


Figure 12: 'sudo ls -la /root' listing files including 3rd.txt.

Root's home directory contained a file named 3rd.txt, which was read (again via sudo/less as needed) and contained the final ingredient:

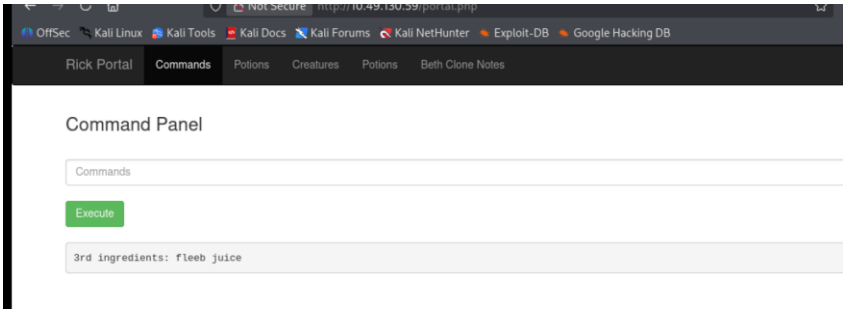


Figure 13: Final ingredient retrieved — '3rd ingredients: fleeb juice'.

- **Flag 3 (Root flag): "fleeb juice"**

4. Summary of Flags Captured

Ingredient 1 (Web)	mr. meeseek hair — /var/www/html/Sup3rS3cretPickl3Ingred.txt
Ingredient 2 (User)	1 jerry tear — /home/rick/"second ingredients"
Ingredient 3 (Root)	fleeb juice — /root/3rd.txt (via sudo)

5. Vulnerabilities Identified

5.1 Information Disclosure

Sensitive data (a valid username in an HTML source-code comment, and a valid password string in a publicly accessible robots.txt file) was exposed to any unauthenticated user, allowing straightforward credential harvesting without any exploitation skill.

5.2 OS Command Injection

Once authenticated, the "Command Panel" on portal.php passed user input directly to the operating system shell with no proper sanitisation (only a partial blacklist, since 'cat' was blocked but 'less' was not). This allowed full arbitrary command execution as the www-data user, and combined with a misconfigured sudo policy, led to full root compromise of the host.

5.3 Risk Rating

This is a lab environment created for training purposes (TryHackMe — Pickle Rick), so there is no corresponding public CVE identifier; the CVSS scores below are estimated by the author to reflect the equivalent real-world risk of each finding.

Vulnerability	CVSS 3.1	Severity	Location
Information Disclosure (sensitive data exposed in HTML source & robots.txt)	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)	Medium	HTTP root page source, /robots.txt
OS Command Injection (unsanitised command panel on portal.php)	9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)	Critical	/portal.php – Command Panel input field

6. Recommendations

- Remove credentials, comments, and other sensitive information from HTML source code and publicly accessible files such as robots.txt.
- Never build command-panel style features that pass raw user input to a system shell; if OS-level actions are required, use a strict allow-list of safe, parameterised operations instead of a command blacklist.
- Apply the principle of least privilege — the web service account (www-data) should not have sudo rights to arbitrary commands.
- Enforce strong, unique credentials and multi-factor authentication on any administrative login page.
- Regularly audit web-accessible directories and files for accidental exposure of sensitive data.

7. Conclusion

The Pickle Rick lab demonstrates how a combination of a low-severity information disclosure issue and a high-severity command injection vulnerability can be chained together to achieve full compromise of a web server, from initial foothold as www-data through to root access. All three objective flags (ingredients) were successfully retrieved, meeting the goal of the exercise.